

Document 2: Simulation Document

Formula-to-R-Code Mapping for Bartlett Corrections

Maresh Pravinsinh Solanki

May 21, 2026

Contents

1	Introduction	2
1.1	Software Platform	2
1.2	Overview of the Four Methods	2
2	Simulation Design Overview	3
2.1	Factorial Design Structure	3
2.2	Replication and Significance Level	3
3	Model Specifications with R Code	4
3.1	Model 1: Simple CFA ($p = 9, t = 21, df = 24$)	4
3.2	Model 2: Medium CFA ($p = 12, t = 27, df = 51$)	5
3.3	Model 3: Full SEM ($p = 9, t = 22, df = 23$)	6
4	Method-by-Method: Formula to R Code Mapping	6
4.1	Method 1: Uncorrected ML	7
4.2	Method 2: Swain (1975)	8
4.3	Method 3: Bartlett EFA (1950)	9
4.4	Method 4: Yuan (2015)	10
5	Data Generation: Normal vs. Non-Normal	11
5.1	Normal Data Generation	11
5.2	Non-Normal Data Generation (Skew = 2, Kurtosis = 3)	12
6	Simulation Loop Structure	13
6.1	Seed Management for Reproducibility	15
6.2	Convergence Handling	15
7	Rejection Rate Calculation	15
7.1	Bradley (1978) Robustness Criterion	16
7.2	Summary Table Construction	17
8	Summary and Reproducibility Notes	17

1 Introduction

The primary purpose of this document is to provide a transparent and reproducible bridge between the mathematical theory of small-sample chi-square corrections and their concrete implementation in the R programming language. While theoretical papers present correction formulae in abstract notation, simulation studies must translate every symbol—sample size, degrees of freedom, number of parameters, and observed variables—into executable code. Any ambiguity in this translation introduces the risk that the simulation no longer reflects the method it purports to evaluate, undermining the validity of all subsequent conclusions.

To eliminate this risk, each section of this document presents the correction method’s theoretical formula side by side with the corresponding R code implementation. For every correction factor, we first state the mathematical definition in standard notation, then show exactly how the R packages `lavaan` (version 0.6–17 or later) and `semTools` (version 0.5–7 or later) compute the corrected test statistic, and finally provide a manual reimplementaion that makes every intermediate quantity explicit. This three-layered approach—theory, package output, and manual verification—ensures that the reader can confirm the simulation faithfully implements each method.

1.1 Software Platform

All simulation work is conducted in the R statistical computing environment (version 4.3.0 or later). Structural equation models are specified, estimated, and tested using the `lavaan` package, which provides a unified syntax for confirmatory factor analysis (CFA) and full structural equation modelling (SEM). The small-sample chi-square corrections are obtained through the `chisqSmallN()` function from the `semTools` package, which implements four distinct correction methods: the uncorrected maximum likelihood (ML) test statistic, the Swain (1975) correction, the Bartlett (1950) EFA-style correction, and the Yuan (2015) mean-and-variance correction. Data generation for non-normal conditions relies on the `mvrnonnorm()` function, also provided by `semTools`, which implements the Vale and Maurelli (1983) method for generating multivariate data with user-specified univariate skewness and kurtosis.

1.2 Overview of the Four Methods

The simulation evaluates four chi-square test statistics for model fit, each derived from the standard ML fit function but differing in their finite-sample adjustments. The methods, ordered from least to most aggressive in their correction, are:

- (i) **Uncorrected ML** (T_{ML}): The conventional likelihood-ratio test statistic, which assumes the test statistic follows a central χ^2 distribution with df degrees of freedom. This serves as the baseline against which all corrections are compared.
- (ii) **Swain (1975)** (T_{Swain}): A rescaling correction derived from Bartlett’s expansion of the likelihood-ratio statistic in the presence of nuisance parameters. It adjusts the test statistic by a factor that depends on the total number of parameters and observed variables.

- (iii) **Bartlett EFA (1950)** (T_{Bart}): Originally developed for exploratory factor analysis, this correction divides the ML test statistic by a bias-reduction factor that incorporates the number of factors. Its application to CFA and SEM is an extension that may or may not retain its theoretical justification.
- (iv) **Yuan (2015)** (T_{Yuan}): The most sophisticated correction, which adjusts both the mean and the variance of the reference distribution. It uses an adjusted degrees of freedom df^* that accounts for the additional variability introduced by parameter estimation, yielding a reference $\chi^2(df^*)$ distribution.

2 Simulation Design Overview

The simulation adopts a fully crossed factorial design in which every combination of model complexity, sample size, and distributional condition is evaluated. This design ensures that the effects of each factor—and their interactions—on the Type I error rate can be estimated with precision.

2.1 Factorial Design Structure

The simulation manipulates three independent variables:

- F1. Model complexity** (3 levels): A simple CFA with $p = 9$ observed variables and $t = 21$ free parameters; a medium CFA with $p = 12$ and $t = 27$; and a full SEM with $p = 9$, $t = 22$, and latent regression paths. These models span a range of parameter-to-observation ratios, from the relatively parsimonious (Model 1) to the more complex (Model 2, which has a higher t/p ratio).
- F2. Sample size** (8 levels): $N \in \{25, 50, 75, 100, 150, 300, 500, 1000\}$. This range captures the critical small-sample region ($N \leq 100$), a moderate-sample transition zone ($100 < N \leq 300$), and large-sample conditions ($N > 300$) where asymptotic theory should provide adequate approximations.
- F3. Distributional condition** (2 levels): Multivariate normal data and moderately non-normal data with population univariate skewness of 2.0 and excess kurtosis of 3.0. This permits evaluation of whether the corrections, which were derived under normality, are robust to violations of this assumption.

The fully crossed design yields $3 \times 8 \times 2 = 48$ experimental conditions.

2.2 Replication and Significance Level

Each experimental condition is replicated $R = 5000$ times. This number of replications ensures that the Monte Carlo standard error of the estimated Type I error rate $\hat{\alpha}$ is sufficiently small to permit meaningful distinctions between methods. Specifically, when the true Type I error rate is $\alpha = 0.05$, the standard error is:

$$SE(\hat{\alpha}) = \sqrt{\frac{\alpha(1-\alpha)}{R}} = \sqrt{\frac{0.05 \times 0.95}{5000}} \approx 0.0031, \quad (1)$$

providing a 95% confidence interval width of approximately ± 0.006 around the point estimate. This precision is sufficient for the Bradley (1978) robustness criterion, which requires the empirical rejection rate to fall within the interval $[0.025, 0.075]$ for a nominal $\alpha = 0.05$ test.

The nominal significance level is fixed at $\alpha = 0.05$ throughout the simulation. Each replication generates a data set from the population model (equivalently, from the model-implied covariance matrix $\Sigma(\theta_0)$), fits the same model to the data, extracts the chi-square test statistics for all four methods, and records whether each method rejects the null hypothesis.

3 Model Specifications with R Code

This section provides the complete specification of each of the three models used in the simulation, including the mathematical parameter counting, the `lavaan` model syntax, the population parameter values used for data generation, and the reasoning behind the degrees-of-freedom calculation.

3.1 Model 1: Simple CFA ($p = 9$, $t = 21$, $df = 24$)

Model 1 is a three-factor confirmatory factor analysis model with three indicator variables per factor and one correlated error (factor covariance) between the first and second factors. This model represents the simplest non-trivial CFA structure and is widely used in methodological studies of small-sample behaviour.

Parameter Count for Model 1

The total number of free parameters is computed as follows:

$$t = \underbrace{9}_{\text{factor loadings}} + \underbrace{3}_{\text{factor variances}} + \underbrace{1}_{\text{factor covariance (f1-f2)}} + \underbrace{9}_{\text{residual variances}} - \underbrace{1}_{\text{scale constraint}} \quad (2)$$

$$= 21.$$

The degrees of freedom follow from the standard formula $df = p(p+1)/2 - t = 9 \times 10/2 - 21 = 45 - 21 = 24$.

lavaan Model Syntax --- Model 1

```
1 simple_cfa <- '
2   f1 =~ x1 + x2 + x3
3   f2 =~ x4 + x5 + x6
4   f3 =~ x7 + x8 + x9
5   f1 ~~ f2
6 '
```

The population parameter values used for data generation are shown in Table 1. All factor

loadings are set to 0.7, providing a standardized loading that is typical of well-fitting models in the social sciences. Factor variances are fixed to 1.0, and the covariance between f_1 and f_2 is 0.3, yielding a factor correlation of $\rho = 0.30$. Residual variances are determined by the constraint $\theta_{ii} = 1 - \lambda_i^2 = 1 - 0.49 = 0.51$.

Table 1: Population parameter values for Model 1 (simple CFA).

Parameter Type	Parameter	Value
Factor loadings	$\lambda_{i1}, \lambda_{i2}, \lambda_{i3}$	0.7
Factor variances	$\phi_{11}, \phi_{22}, \phi_{33}$	1.0
Factor covariance	ϕ_{12}	0.3
Residual variances	θ_{11} through θ_{99}	0.51

3.2 Model 2: Medium CFA ($p = 12$, $t = 27$, $df = 51$)

Model 2 extends the simple CFA to four factors, each measured by three indicators, with three of the six possible factor covariances freely estimated. This model increases the number of observed variables and parameters while maintaining a clear factor structure.

Parameter Count for Model 2

The total number of free parameters is:

$$\begin{aligned}
 t &= \underbrace{12}_{\text{loadings}} + \underbrace{4}_{\text{factor var.}} + \underbrace{3}_{\text{factor cov.}} \\
 &+ \underbrace{12}_{\text{residual var.}} - \underbrace{4}_{\text{scale constr.}} \\
 &= 27.
 \end{aligned} \tag{3}$$

The degrees of freedom are $df = p(p+1)/2 - t = 12 \times 13/2 - 27 = 78 - 27 = 51$. The three estimated factor covariances are ϕ_{12} , ϕ_{23} , and ϕ_{34} .

lavaan Model Syntax --- Model 2

```

1 medium_cfa <- '
2   f1 =~ x1 + x2 + x3
3   f2 =~ x4 + x5 + x6
4   f3 =~ x7 + x8 + x9
5   f4 =~ x10 + x11 + x12
6   f1 ~~ f2
7   f2 ~~ f3
8   f3 ~~ f4
9 '
```

Table 2: Population parameter values for Model 2 (medium CFA).

Parameter Type	Parameter	Value
Factor loadings	λ_{i1} through λ_{i4}	0.7
Factor variances	$\phi_{11}, \phi_{22}, \phi_{33}, \phi_{44}$	1.0
Factor covariances	$\phi_{12}, \phi_{23}, \phi_{34}$	0.3
Residual variances	θ_{11} through $\theta_{12,12}$	0.51

3.3 Model 3: Full SEM ($p = 9, t = 22, df = 23$)

Model 3 is a full structural equation model that combines the measurement portion of Model 1 with two structural regression paths: f_3 is regressed on both f_1 and f_2 . This model introduces directional relationships among latent variables, making it a genuine SEM rather than a pure CFA.

Parameter Count for Model 3

The total number of free parameters is:

$$\begin{aligned}
 t &= \underbrace{9}_{\text{loadings}} + \underbrace{3}_{\text{exog. factor var.}} + \underbrace{1}_{\text{factor cov.}} \\
 &+ \underbrace{2}_{\text{regression paths}} + \underbrace{1}_{\text{f3 residual}} + \underbrace{9}_{\text{indicator resid.}} \\
 &- \underbrace{3}_{\text{scale constr.}} \\
 &= 22.
 \end{aligned} \tag{4}$$

The degrees of freedom are $df = p(p+1)/2 - t = 45 - 22 = 23$. Note that the addition of two regression paths and one endogenous residual variance (net +3 parameters) reduces df by 3 relative to Model 1.

lavaan Model Syntax --- Model 3

```

1 full_sem <- '
2   f1 =~ x1 + x2 + x3
3   f2 =~ x4 + x5 + x6
4   f3 =~ x7 + x8 + x9
5   f3 ~ f1 + f2
6   f1 ~~ f2
7 '
```

4 Method-by-Method: Formula to R Code Mapping

This section constitutes the core of the document. For each of the four test statistic methods, we present: (a) the theoretical formula in mathematical notation, (b) the R code that invokes the built-in `semTools` implementation, and (c) an explicit manual calculation that reconstructs

Table 3: Population parameter values for Model 3 (full SEM).

Parameter Type	Parameter	Value
Factor loadings	$\lambda_{i1}, \lambda_{i2}, \lambda_{i3}$	0.7
Exogenous factor variances	ϕ_{11}, ϕ_{22}	1.0
Factor covariance	ϕ_{12}	0.3
Regression coefficients	β_{31}, β_{32}	0.4, 0.3
Endogenous residual variance	ψ_{33}	0.72
Indicator residual variances	θ_{11} through θ_{99}	0.51

the corrected statistic from its constituent parts. The manual calculation serves as a verification tool, allowing the researcher to confirm that the package output matches the intended formula.

4.1 Method 1: Uncorrected ML

The uncorrected maximum likelihood test statistic is the standard likelihood-ratio test for the null hypothesis that the population covariance matrix Σ equals the model-implied covariance matrix $\Sigma(\theta)$. Under the null hypothesis and with sufficiently large sample size, T_{ML} is asymptotically distributed as a central χ^2 random variable with $df = p(p+1)/2 - t$ degrees of freedom.

Theoretical Formula — Uncorrected ML

$$T_{ML} = (n - 1) F_{ML}(\hat{\theta}), \quad (5)$$

where $n = N$ is the sample size, $F_{ML}(\hat{\theta})$ is the minimum of the ML discrepancy function, and under H_0 :

$$T_{ML} \xrightarrow{d} \chi^2(df). \quad (6)$$

The model is rejected when $p = P(\chi^2(df) > T_{ML}) < \alpha$.

R Implementation --- Uncorrected ML

```

1 # Extract the ML chi-square and its p-value from a lavaan fit
2 T_ML <- fitMeasures(fit, "chisq")
3 p_ML <- fitMeasures(fit, "pvalue")
4 df <- fitMeasures(fit, "df")
5
6 # Rejection decision
7 reject_ML <- p_ML < 0.05

```

The `fitMeasures()` function extracts a wide range of fit indices from a fitted `lavaan` object. The string "chisq" returns the ML test statistic multiplied by the scaling factor (which is 1.0 for ML estimation), and "pvalue" returns the corresponding upper-tail probability from the $\chi^2(df)$ distribution. No small-sample adjustment is applied; this method assumes that the asymptotic reference distribution is adequate regardless of sample size.

4.2 Method 2: Swain (1975)

The Swain (1975) correction addresses the positive bias of T_{ML} in small samples by multiplying the test statistic by a ratio of two linear functions of n , t , and p . The correction is derived from the higher-order expansion of the expected value of the likelihood-ratio statistic and aims to produce a test statistic whose expectation more closely matches the nominal degrees of freedom.

Theoretical Formula — Swain (1975)

$$T_{\text{Swain}} = T_{\text{ML}} \times \frac{2n - 2t - 2p - 1}{2t + 2p + 3}, \quad (7)$$

where $n = N - 1$ (the sample size minus one), t is the number of free parameters, and p is the number of observed variables. The corrected statistic is compared to the same $\chi^2(df)$ reference distribution.

R Implementation --- Swain (1975)

```
1 # Using semTools: one-line extraction
2 smallN_swain <- chisqSmallN(fit, smallN.method = "swain")
3 T_swain <- smallN_swain$chisq
4 p_swain <- smallN_swain$p
5 reject_swain <- p_swain < 0.05
6
7 # ---- Manual verification ----
8 n <- N - 1
9 p <- lavInspect(fit, "ntotal") # number of observed variables
10 t <- lavInspect(fit, "npar")   # number of free parameters
11
12 # Swain correction factor (ratio form)
13 c_swain <- (2*n - 2*t - 2*p - 1) / (2*t + 2*p + 3)
14
15 # Equivalently, the correction can be written as a divisor:
16 # T_swain = T_ML / (1 + 1/c_swain)
17 T_swain_manual <- T_ML / (1 + 1/c_swain)
18
19 # Verify that both computations agree
20 stopifnot(abs(T_swain - T_swain_manual) < 1e-10)
```

The manual verification is critical for two reasons. First, it confirms that the `semTools` implementation uses the same definition of n , t , and p as the theoretical formula. Second, it exposes potential edge cases, such as when the correction factor becomes negative (which can occur when n is very small relative to t and p). The `stopifnot()` assertion should be inserted during development and removed before the production simulation run to avoid unnecessary overhead across $R = 5000$ replications.

4.3 Method 3: Bartlett EFA (1950)

The Bartlett (1950) correction was originally derived for exploratory factor analysis, where the number of factors q is a known parameter of the correction. Its application to confirmatory models requires the user to specify the number of factors, which is straightforward for CFA models (where q equals the number of latent variables) but requires careful interpretation for full SEM models (where the number of “factors” may be ambiguous if some latent variables are purely endogenous).

Theoretical Formula — Bartlett (1950) EFA Correction

The Bartlett correction factor is:

$$c_{\text{Bart}} = \frac{n - \frac{2p + 4q + 1}{2}}{n - 1}, \quad (8)$$

where q is the number of common factors. The corrected test statistic is:

$$T_{\text{Bart}} = \frac{T_{\text{ML}}}{c_{\text{Bart}}}, \quad (9)$$

and T_{Bart} is compared to $\chi^2(df)$.

R Implementation --- Bartlett (1950)

```
1 # Using semTools: specify the number of factors
2 smallN_bart <- chisqSmallN(fit, smallN.method = "bartlett")
3 T_bart <- smallN_bart$chisq
4 p_bart <- smallN_bart$p
5 reject_bart <- p_bart < 0.05
6
7 # ---- Manual verification ----
8 n <- N - 1
9 p <- lavInspect(fit, "ntotal") # observed variables
10 q <- 3 # number of factors (Model 1 and Model 3)
11
12 # Bartlett correction factor
13 c_bart <- (n - (2*p + 4*q + 1)/2) / (n - 1)
14
15 # Corrected test statistic
16 T_bart_manual <- T_ML / c_bart
17
18 # Verify agreement
19 stopifnot(abs(T_bart - T_bart_manual) < 1e-10)
```

An important subtlety in the Bartlett correction is the definition of q . For Model 1 (simple CFA), $q = 3$ is unambiguous. For Model 2 (medium CFA), $q = 4$. For Model 3 (full SEM), all three latent variables function as common factors in the measurement model, so $q = 3$ is the appropriate choice. However, if one interprets q as the number of *exogenous* common factors,

the value would be $q = 2$ for Model 3. The simulation uses $q = 3$ to be consistent with the total number of latent variables, as this is the convention adopted by `semTools`.

4.4 Method 4: Yuan (2015)

The Yuan (2015) correction is the most sophisticated of the four methods, as it adjusts not only the mean but also the variance of the reference distribution. The corrected statistic is compared to a χ^2 distribution with *adjusted* degrees of freedom df^* , which is typically smaller than the nominal df . This dual adjustment produces a reference distribution whose first two moments more closely match those of the finite-sample distribution of the test statistic.

Theoretical Formula — Yuan (2015)

The corrected test statistic and adjusted degrees of freedom are:

$$T_{\text{Yuan}} = T_{\text{ML}} \times \frac{df^*}{df}, \quad (10)$$

$$df^* = df - \frac{2t + 1}{n}, \quad (11)$$

where $n = N - 1$, t is the number of free parameters, and $df = p(p + 1)/2 - t$ is the nominal degrees of freedom. The corrected statistic is compared to $\chi^2(df^*)$ rather than $\chi^2(df)$.

R Implementation --- Yuan (2015)

```

1 # Using semTools
2 smallN_yuan <- chisqSmallN(fit, smallN.method = "yuan.2015")
3 T_yuan <- smallN_yuan$chisq
4 p_yuan <- smallN_yuan$p
5 reject_yuan <- p_yuan < 0.05
6
7 # ---- Manual verification ----
8 n <- N - 1
9 t <- lavInspect(fit, "npar")
10 p <- lavInspect(fit, "ntotal")
11 df_nominal <- p*(p+1)/2 - t
12
13 # Adjusted degrees of freedom
14 df_star <- df_nominal - (2*t + 1) / n
15
16 # Rescaling factor
17 scale_factor <- df_star / df_nominal
18
19 # Corrected test statistic
20 T_yuan_manual <- T_ML * scale_factor
21
22 # p-value under chi-squared(df_star)
23 p_yuan_manual <- 1 - pchisq(T_yuan_manual, df = df_star)

```

```

24 |
25 | # Verify agreement
26 | stopifnot(abs(T_yuan - T_yuan_manual) < 1e-10)
27 | stopifnot(abs(p_yuan - p_yuan_manual) < 1e-10)

```

The Yuan correction has a distinctive property: it adjusts the reference distribution rather than (only) the test statistic. This means that the p -value is computed from $\chi^2(df^*)$ rather than $\chi^2(df)$, which affects both the location and the tail behaviour of the null distribution. For small n and large t , the reduction in degrees of freedom can be substantial, leading to a more conservative test. For example, with Model 1 ($t = 21$, $df = 24$) and $N = 50$ ($n = 49$):

$$df^* = 24 - \frac{2 \times 21 + 1}{49} = 24 - \frac{43}{49} \approx 23.12. \quad (12)$$

5 Data Generation: Normal vs. Non-Normal

The simulation requires two types of data: multivariate normal data, which satisfies the distributional assumptions underlying all four correction methods, and multivariate non-normal data, which tests the robustness of the corrections when these assumptions are violated. Both types are generated from the *same* population covariance matrix $\Sigma(\theta_0)$, ensuring that any differences in Type I error rates between the two conditions are attributable to distributional shape rather than to differences in model fit.

5.1 Normal Data Generation

Under multivariate normality, the population data-generating process is:

$$\mathbf{x} \sim \mathcal{N}_p(\mathbf{0}, \Sigma(\theta_0)), \quad (13)$$

where $\Sigma(\theta_0)$ is the model-implied covariance matrix evaluated at the population parameter values. The mean vector is $\mathbf{0}$ without loss of generality, because the ML fit function depends only on the sample covariance matrix $\mathbf{S}$, which is location-invariant.

```

R Code --- Normal Data Generation
1 | # Generate data from the population model using lavaan
2 | library(lavaan)
3 |
4 | # Fit the model to obtain the implied covariance matrix
5 | fit_pop <- lavaan(model_syntax, sample.cov = NULL,
6 |                  sample.nobs = 1e6, sample.mean = rep(0, p))
7 | Sigma_pop <- lavInspect(fit_pop, "Sigma.hat")
8 |
9 | # Generate a sample of size N
10 | set.seed(seed_vector[rep])
11 | data_norm <- MASS::mvrnorm(N, mu = rep(0, p), Sigma = Sigma_pop)

```

```

12 colnames(data_norm) <- paste0("x", 1:p)
13
14 # Alternatively, use simulateData for convenience:
15 data_norm <- simulateData(model_syntax, sample.nobs = N,
16                           empirical = FALSE)

```

The `simulateData()` function is the preferred approach because it automatically computes the model-implied covariance matrix from the `lavaan` model syntax, eliminating the need for a preliminary fitting step. The argument `empirical = FALSE` ensures that the generated data are a random sample from the specified distribution rather than a sample whose sample covariance matrix exactly matches the population covariance matrix.

5.2 Non-Normal Data Generation (Skew = 2, Kurtosis = 3)

Non-normal data are generated using the method of Vale and Maurelli (1983), which constructs a third-order polynomial transformation of standard normal variables to achieve specified univariate skewness and excess kurtosis while preserving a target correlation (or covariance) structure. The method is implemented in the `mvnrnonnorm()` function from `semTools`.

Vale–Maurelli (1983) Method

Let $\mathbf{z} \sim \mathcal{N}_p(\mathbf{0}, \mathbf{R}^*)$, where $\mathbf{R}^*$ is an intermediate correlation matrix. Each component is transformed as:

$$x_i = a + b z_i + c z_i^2 + d z_i^3, \quad (14)$$

where the coefficients $\{a, b, c, d\}$ are chosen so that $\text{Skew}(x_i) = \gamma_1$ and $\text{Kurt}(x_i) = \gamma_2 + 3$ (as specified by the user), and $\mathbf{R}^*$ is computed so that the *product-moment* correlation between x_i and x_j equals the corresponding element of the target matrix.

R Code --- Non-Normal Data Generation

```

1 library(semTools)
2 library(lavaan)
3
4 # Step 1: Obtain the population implied covariance matrix
5 fit_pop <- lavaan(model_syntax, sample.cov = NULL,
6                  sample.nobs = 1e6, sample.mean = rep(0, p))
7 Sigma_pop <- lavInspect(fit_pop, "Sigma.hat")
8 mu <- rep(0, p)
9
10 # Step 2: Generate non-normal data using Vale-Maurelli method
11 set.seed(seed_vector[rep])
12 data_nonnorm <- mvnrnonnorm(N, mu, Sigma_pop,
13                             skewness = rep(2, p),
14                             kurtosis = rep(3, p))
15 colnames(data_nonnorm) <- paste0("x", 1:p)

```

The choice of $\gamma_1 = 2$ (moderate positive skewness) and $\gamma_2 = 3$ (moderate positive excess kurtosis, yielding a total kurtosis of 6) is intentionally set in the moderate range. This level of non-normality is typical of real-world data in the social and behavioural sciences (Micceri, 1989), where mild to moderate departures from normality are the rule rather than the exception. More extreme non-normality (e.g., $\gamma_1 > 3$) would push the conditions beyond what the corrections were designed to handle and could lead to convergence failures, particularly at small sample sizes.

6 Simulation Loop Structure

This section presents the pseudocode and structural logic for the complete simulation loop. The design follows best practices for Monte Carlo simulation studies in structural equation modelling (Bandalos, 2006; Paxton, Curran, Bollen, Kirby, & Ferron, 2001), including proper seed management for reproducibility, convergence monitoring, and structured storage of results.

Complete Simulation Pseudocode

```

1 library(lavaan)
2 library(semTools)
3
4 # ---- Configuration ----
5 N_vec      <- c(25, 50, 75, 100, 150, 300, 500, 1000)
6 R          <- 5000
7 alpha      <- 0.05
8 models     <- list(simple_cfa, medium_cfa, full_sem)
9 dist_cond  <- c("normal", "nonnormal")
10 q_vec      <- c(3, 4, 3) # number of factors per model
11
12 # Seed management: one seed per condition
13 set.seed(20240101)
14 all_seeds <- sample.int(1e7, size = length(models) *
15                           length(N_vec) * length(dist_cond) * R)
16
17 # ---- Results storage ----
18 results <- array(0,
19   dim = c(length(models), length(N_vec),
20           length(dist_cond), 4), # 4 methods
21   dimnames = list(
22     model = c("M1_CFA", "M2_CFA", "M3_SEM"),
23     N = as.character(N_vec),
24     dist = c("normal", "nonnormal"),
25     method = c("ML", "Swain", "Bartlett", "Yuan")))
26
27 # ---- Main simulation loop ----
28 seed_idx <- 1
29 for (m in seq_along(models)) {
30   for (n_idx in seq_along(N_vec)) {

```

```

31 N <- N_vec[n_idx]
32 for (d in seq_along(dist_cond)) {
33   rej_count <- rep(0L, 4) # counters for 4 methods
34   converged <- 0L
35
36   for (rep in 1:R) {
37     # --- Generate data ---
38     set.seed(all_seeds[seed_idx]); seed_idx <- seed_idx + 1
39
40     if (dist_cond[d] == "normal") {
41       dat <- simulateData(models[[m]], sample.nobs = N,
42                             empirical = FALSE)
43     } else {
44       Sigma_pop <- lavaan(models[[m]], sample.nobs = 1e6) |>
45         lavInspect("Sigma.hat")
46       dat <- mvrnonnorm(N, rep(0, ncol(Sigma_pop)), Sigma_pop,
47                           skewness = rep(2, ncol(Sigma_pop)),
48                           kurtosis = rep(3, ncol(Sigma_pop)))
49       colnames(dat) <- paste0("x", 1:ncol(Sigma_pop))
50     }
51
52     # --- Fit model ---
53     fit <- tryCatch(
54       lavaan(models[[m]], data = dat),
55       error = function(e) NULL)
56
57     if (is.null(fit)) next # skip non-converged
58     converged <- converged + 1L
59
60     # --- Extract test statistics ---
61     T_ML <- fitMeasures(fit, "chisq")
62
63     # Method 1: Uncorrected ML
64     rej_count[1] <- rej_count[1] +
65       (fitMeasures(fit, "pvalue") < alpha)
66
67     # Method 2: Swain
68     sw <- chisqSmallN(fit, "swain")
69     rej_count[2] <- rej_count[2] + (sw$p < alpha)
70
71     # Method 3: Bartlett
72     bt <- chisqSmallN(fit, "bartlett")
73     rej_count[3] <- rej_count[3] + (bt$p < alpha)
74
75     # Method 4: Yuan
76     yu <- chisqSmallN(fit, "yuan.2015")

```

```

77     rej_count[4] <- rej_count[4] + (yu$p < alpha)
78   } # end replications
79
80   # --- Store rejection rates ---
81   results[m, n_idx, d, ] <- rej_count / converged
82 } # end distribution
83 } # end sample size
84 } # end models

```

6.1 Seed Management for Reproducibility

Reproducibility is a cornerstone of credible simulation research. The simulation uses a pre-generated vector of unique seeds, created at the beginning of the program from a single master seed (e.g., 20240101). Each replication receives its own seed, ensuring that: (a) the results are identical across runs on the same platform, (b) the order of replications can be changed (e.g., for parallelisation) without altering the generated data sets, and (c) any specific replication can be reproduced by setting the corresponding seed and re-running the data generation and analysis steps.

6.2 Convergence Handling

Convergence failures are an expected occurrence in small-sample simulations, particularly when N is small relative to the number of parameters. The simulation adopts a conservative approach: non-converged replications are excluded from the rejection rate calculation rather than counted as non-rejections. The convergence rate itself is recorded as a supplementary outcome, because a correction method that achieves good Type I error control at the cost of frequent convergence failures is of limited practical utility. Convergence is assessed through **lavaan**'s internal optimisation routines, which check both the first-order gradient and the parameter change criterion at each iteration.

7 Rejection Rate Calculation

The Type I error rate for each combination of model, sample size, distributional condition, and correction method is estimated as the proportion of replications in which the method rejects the true null hypothesis.

Rejection Rate Formula

The estimated Type I error rate is:

$$\hat{\alpha} = \frac{1}{R_{\text{conv}}} \sum_{r=1}^{R_{\text{conv}}} I(p_r < \alpha), \quad (15)$$

where R_{conv} is the number of converged replications (out of the total $R = 5000$), p_r is the p -value from replication r , and $I(\cdot)$ is the indicator function. The Monte Carlo standard error is:

$$\text{SE}(\hat{\alpha}) = \sqrt{\frac{\hat{\alpha}(1 - \hat{\alpha})}{R_{\text{conv}}}}. \quad (16)$$

7.1 Bradley (1978) Robustness Criterion

Following Bradley (1978), a test is classified as maintaining adequate control of the Type I error rate if its empirical rejection rate falls within the interval:

$$\hat{\alpha} \in [0.5\alpha, 1.5\alpha] = [0.025, 0.075] \quad \text{for } \alpha = 0.05. \quad (17)$$

Methods whose rejection rates fall outside this interval are classified as *liberal* ($\hat{\alpha} > 0.075$) or *conservative* ($\hat{\alpha} < 0.025$). A liberal test rejects the true null hypothesis too frequently, inflating the risk of false positive conclusions. A conservative test has reduced statistical power, increasing the risk of false negative conclusions.

R Code --- Rejection Rate Classification

```

1 # Compute rejection rate and its standard error
2 compute_rejection <- function(reject_vector, alpha = 0.05) {
3   R_conv <- length(reject_vector)
4   alpha_hat <- mean(reject_vector)
5   se <- sqrt(alpha_hat * (1 - alpha_hat) / R_conv)
6
7   # 95% confidence interval (normal approximation)
8   ci_lower <- alpha_hat - 1.96 * se
9   ci_upper <- alpha_hat + 1.96 * se
10
11   # Bradley classification
12   lower_bradley <- 0.5 * alpha # 0.025
13   upper_bradley <- 1.5 * alpha # 0.075
14
15   if (alpha_hat < lower_bradley) {
16     classification <- "Conservative"
17   } else if (alpha_hat > upper_bradley) {
18     classification <- "Liberal"
19   } else {
20     classification <- "Robust"
21   }
22
23   list(
24     alpha_hat = alpha_hat,
25     se = se,
26     ci = c(ci_lower, ci_upper),

```



```

27     classification = classification,
28     R_converged    = R_conv
29   )
30 }
31
32 # Example usage:
33 # result <- compute_rejection(reject_ML_vector, alpha = 0.05)
34 # print(result)

```

7.2 Summary Table Construction

After computing the rejection rates for all 48 conditions and four methods, the results are organised into summary tables that facilitate comparison across methods and conditions. A typical summary table presents the rejection rate (with standard error in parentheses) for each combination of sample size and method, separately for the normal and non-normal distributional conditions. Methods that satisfy the Bradley criterion are highlighted in bold or marked with a symbol, providing a visual summary of which methods achieve adequate Type I error control under each condition.

8 Summary and Reproducibility Notes

This document has presented a complete mapping between the theoretical formulae for four small-sample chi-square correction methods and their R code implementations. The mapping covers every computational step, from model specification through data generation, model fitting, test statistic extraction, and rejection rate classification. By making each translation explicit, we ensure that the simulation faithfully implements the intended methods and that the results can be independently verified.

The complete simulation code, including the model specifications, data generation routines, fitting procedures, and analysis scripts, is provided in the accompanying R script files. The master configuration file (`01_config.R`) defines all simulation parameters, and the main simulation loop (`02_run_simulation.R`) implements the pseudocode presented in Section 6. Results are aggregated and analysed in `03_analysis.R`, and the final report is generated by `04_report.R`.

To reproduce the simulation from scratch, the user needs only R (version 4.3.0 or later) with the packages `lavaan`, `semTools`, and `MASS` installed. The total computation time depends on the available hardware but is estimated at approximately 8–12 hours on a modern multi-core workstation when the simulation is run sequentially. Parallelisation across conditions can reduce this time proportionally to the number of available cores.